SOFTTRACKER

Manual de uso de ponta a ponta

Instalação, configuração de um projeto, rastreabilidade por grafo, painel e validação comportamental

Para quem?

Para quem deseja acompanhar um único projeto de software por configuração, relacionando requisitos e critérios de aceitação ao código e, quando configurado, a testes automatizados reais.

> **Importante sobre esta versão**
>
> O pacote limpo não inclui uma aplicação-alvo nem uma especificação pronta. Você escolhe o projeto e a especificação durante a configuração inicial. As capturas de tela deste manual mostram um painel de uma execução anterior e servem apenas para explicar a interface; os números e nomes exibidos nelas não vêm com o pacote limpo.

Versão do manual: 1.0 • Outubro de 2026

1. O que o SoftTracker faz

O SoftTracker acompanha a relação entre a especificação de um produto e o código de um único projeto ativo. Ele indexa elementos de código, procura evidências para cada critério e organiza as relações em um grafo. Quando a integração de testes está configurada, também executa testes e mantém os resultados separados da análise estática.

Parte	Para que serve
Especificação	Descreve requisitos, casos de uso, tarefas, dependências e critérios de aceitação.
Indexador	Localiza arquivos, classes, funções, rotas e outros elementos detectáveis.
Motor de rastreabilidade	Relaciona cada critério às evidências encontradas e mantém o estado atualizado.
Grafo	Representa as relações entre requisitos, tarefas, critérios, elementos de código e, se configurado, testes.
Executor de testes	Executa o comando de teste explicitamente configurado e interpreta relatórios JUnit XML.
Dashboard	Mostra progresso, tarefas disponíveis, dependências, grafo de rastreabilidade e stack detectada.

O que ele não garante sozinho

- Uma correspondência de nome, rota ou trecho é uma evidência estática, não uma prova de que o comportamento funciona.
- A análise de chamadas entre funções é aproximada e pode não resolver por completo recursos dinâmicos, injeção de dependência ou comportamentos distribuídos por vários arquivos.
- A comprovação comportamental depende de testes reais associados aos critérios corretos. Sem testes configurados, o SoftTracker não os executa automaticamente.

Caminho recomendado

1. Instale o Python e extraia o pacote do SoftTracker.
2. Escolha a pasta do código e prepare uma especificação YAML ou JSON do mesmo projeto.
3. Configure o alvo com `init` e valide o manifesto.
4. Execute `sync` e `status`; revise evidências duvidosas.
5. Abra o dashboard e use o grafo para entender o que está feito e o que bloqueia as próximas tarefas.
6. Configure e execute testes comportamentais se quiser comprovar o funcionamento real.

2. Instalação

2.1. Requisitos

- Windows, macOS ou Linux com terminal disponível.
- Python 3.9 ou superior. A versão do pacote foi exercitada com Python 3.13.5; a faixa inteira do Python 3.9+ não foi testada integralmente. Para um ambiente novo, prefira uma versão recente do Python 3.
- Navegador moderno para abrir o dashboard local.
- PyYAML somente se a especificação escolhida estiver em YAML. Especificações JSON podem ser lidas sem essa dependência.
- Node.js, Java, Maven ou outra ferramenta só são necessários se o projeto-alvo ou o comando de testes daquele projeto depender deles; não são requisitos universais do SoftTracker.

2.2. Instalar no Windows

1. Instale Python pelo site oficial: https://www.python.org/downloads/windows/. Durante a instalação, habilite a opção de adicionar Python ao PATH, quando ela estiver disponível.
2. Abra PowerShell ou Prompt de Comando e confirme a instalação:

```
py --version
py -m pip --version
```

Se o comando `py` não existir, tente `python --version`. Use o comando que funcionar nos passos seguintes. Nos exemplos deste manual, `py` é o lançador do Windows; em macOS/Linux, normalmente substitua-o por `python3`.

2.3. Extrair o SoftTracker

1. Baixe o ZIP genérico do SoftTracker e extraia a pasta `SoftTracker` em um local estável, por exemplo `C:/Ferramentas/SoftTracker`.
2. Não extraia o SoftTracker dentro da pasta do projeto que ele vai monitorar. Mantenha a ferramenta e a aplicação-alvo em diretórios separados.
3. Abra o terminal e entre na pasta extraída. Exemplo:

```
cd C:\Ferramentas\SoftTracker
```

2.4. Instalar suporte a YAML (se necessário)

```
py -m pip install PyYAML
```

Se você optar por especificação `.json`, pode pular este passo. Não é necessário instalar bibliotecas de visualização ou um banco de dados separado para abrir o dashboard.

3. Configurar seu primeiro projeto

> **Escolha com cuidado o diretório do alvo**
>
> Use a pasta raiz do projeto que realmente deseja acompanhar - a pasta que contém o código-fonte e, normalmente, os arquivos de build/configuração. Não selecione a pasta do próprio SoftTracker e não aponte para um diretório que contenha vários projetos diferentes.

3.1. Separe os diretórios

```
C:\Ferramentas\SoftTracker\  # ferramenta
D:\App\                        # projeto-alvo
D:\Spec\app.yml               # especificação
```

A especificação pode estar dentro ou fora do projeto-alvo. Guardá-la em `docs` pode ser conveniente, mas verifique se o indexador não vai tratar documentos de especificação como código-fonte relevante; em geral, a especificação pode ficar fora da pasta monitorada.

3.2. Criar a configuração

Na pasta do SoftTracker, execute `init`. Substitua os caminhos abaixo pelos seus caminhos reais. O exemplo usa PowerShell/Windows; no macOS/Linux, troque pelos caminhos correspondentes.

```
py harness.py init --projeto "D:\App" --spec "D:\Spec\app.yml"
```

O comando cria `harness.json` com um único diretório ativo e os arquivos de especificação selecionados. Os caminhos informados são resolvidos na configuração. Você pode fornecer mais de uma especificação após `--spec`, desde que todas descrevam o mesmo projeto-alvo.

3.3. Validar antes de analisar

```
py harness.py validar
```

Corrija todo erro apontado antes de seguir. Avisos também merecem atenção: podem indicar critérios vagos, critérios sem artefatos determinísticos ou configurações incompletas de testes.

3.4. Primeiro status

```
py harness.py sync
py harness.py status --resumo
```

`sync` atualiza o índice, as evidências e o estado persistido. `status --resumo` mostra uma visão compacta; `status --json` é útil para diagnóstico ou integração com outras ferramentas.

Lembrete: a configuração padrão acompanha um único alvo ativo. A troca de alvo limpa o estado gerado padrão; para projetos paralelos, mantenha cópias separadas do SoftTracker.

4. Preparar a especificação

A qualidade da rastreabilidade depende da especificação. Dê IDs estáveis a requisitos, casos de uso, tarefas e critérios. Cada critério deve descrever um resultado observável e verificável, não apenas uma intenção genérica.

4.1. Exemplo mínimo em YAML

```yaml
projeto:
  nome: "Minha Aplicacao"

requisitos:
  - id: RF001
    titulo: "Permitir cadastrar usuario"
    tipo: funcional

casos_de_uso:
  - id: UC001
    titulo: "Cadastrar usuario"
    requisitos: [RF001]

tasks:
  - id: T001
    titulo: "Implementar cadastro de usuario"
    casos_de_uso: [UC001]
    depende_de: []
    criterios:
      - id: C1
        descricao: "A API aceita um nome e cria um usuario"
        artefato:
          tipo: funcao
          padrao: "criarUsuario"
```

4.2. Como escrever bons critérios

Evite	Prefira
"O cadastro deve ser robusto"	"Uma requisição válida cria um usuário e retorna HTTP 201"
"Implementar segurança"	"Requisição sem token retorna HTTP 401"
"A função existe"	"Quando o e-mail já existe, a API retorna HTTP 409 e não cria um segundo usuário"

4.3. Artefatos e dependências

O campo `artefato` pode indicar pistas de código, como `rota`, `funcao`, `classe`, `arquivo`, `simbolo` ou `conteudo`. O `padrao` precisa ser específico o suficiente para localizar o elemento certo. Para evidência de conteúdo que pertença a um método específico, use o campo `escopo` quando aplicável.

Use `depende_de` com IDs de outras tarefas para representar a ordem de implementação. Evite criar dependências circulares. Se for necessário ignorar pastas geradas ou enormes, verifique o campo de nível superior `ignorar` aceito na especificação e inclua os diretórios que não devem ser indexados, por exemplo `node_modules`, `dist` ou `build`, conforme seu projeto.

Mantenha os IDs de tarefa e critério estáveis: eles preservam vínculos, rastreabilidade e a associação entre testes e critérios ao longo do tempo.

5. Comandos do dia a dia

Comando	Quando usar
`py harness.py validar`	Validar a estrutura da especificação, IDs, dependências e configuração dos testes.
`py harness.py sync`	Atualizar o índice, grafo e evidências depois de mudanças.
`py harness.py status --resumo`	Ver rapidamente o progresso geral. O comando atualiza a análise antes de exibir o status.
`py harness.py status --json`	Inspecionar o estado completo em formato JSON.
`py harness.py proxima`	Listar tarefas que já podem começar, respeitando dependências.
`py harness.py go T001`	Selecionar uma tarefa; o SoftTracker pode alertar sobre pré-requisitos pendentes.
`py harness.py pendentes`	Listar critérios que precisam de julgamento ou de vínculo manual.
`py harness.py trecho ID_DO_NO`	Mostrar o trecho de código associado a um nó do índice.
`py harness.py link T001/C1 ID_DO_NO`	Registrar manualmente que um critério está atendido por um nó existente.
`py harness.py nao-feito T001/C1`	Registrar que os candidatos atuais não atendem ao critério pendente.
`py harness.py unlink T001/C1`	Remover o vínculo/julgamento manual do critério.
`py harness.py painel`	Abrir o dashboard local.

Como interpretar os estados

Estado estático	Significado
Feita	Todos os critérios foram atendidos segundo as regras e evidências configuradas.
Parcial	Parte dos critérios tem evidência, mas há critérios pendentes ou sem confirmação.
Não feita / sem evidência	A análise não confirmou critérios suficientes para considerar a tarefa implementada.
Revalidar	Uma evidência previamente usada mudou e precisa ser reavaliada.

Os estados comportamentais dos testes ficam separados: aprovado, falhou, desatualizado, não executado ou incompleto. Um critério pode ter código identificado e, ainda assim, ter um teste falhando.

6. Conhecendo o dashboard: visão geral

A tela principal resume o progresso e organiza a navegação. A imagem abaixo é uma captura real de uma execução anterior; ela mostra o projeto de exemplo “Gerenciador simples de tarefas”. A versão genérica limpa não inclui esses dados. Depois de configurar seu próprio alvo, os títulos, tarefas, contagens e tecnologias exibidos serão os do seu projeto.

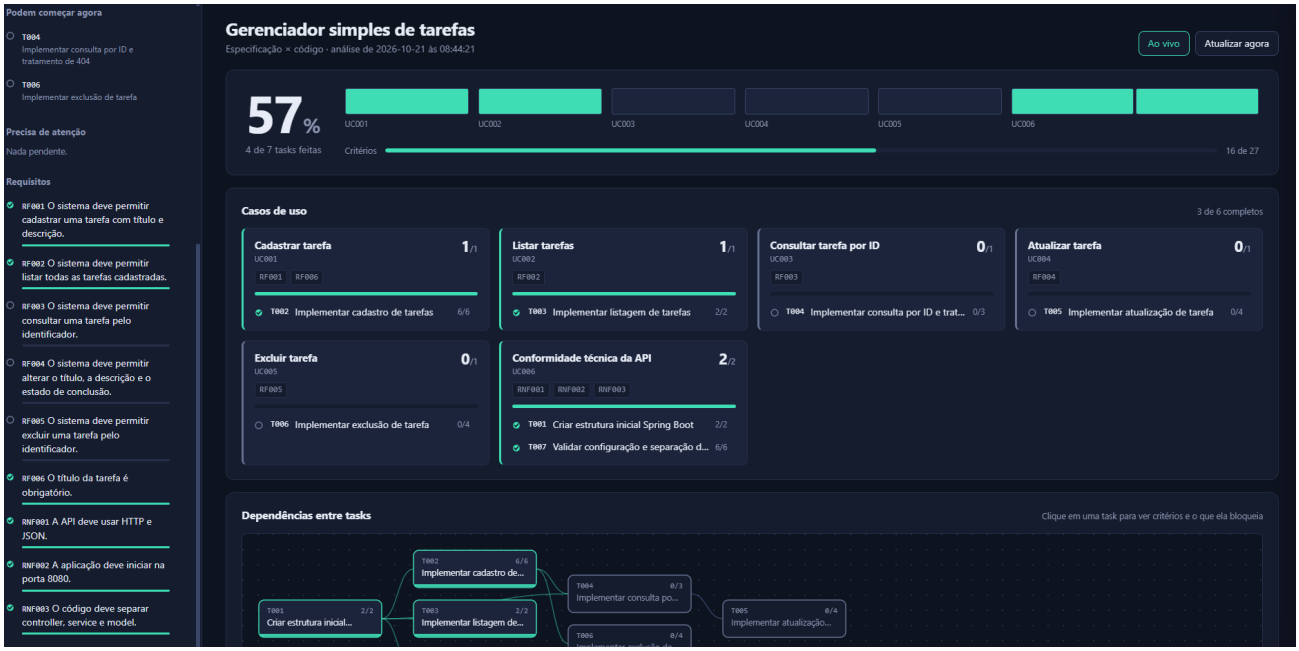


Figura 1 — Visão geral do dashboard: progresso, casos de uso e início do grafo de dependências.

Como ler esta tela

- Barra lateral: lista tarefas que podem começar, itens que exigem atenção e requisitos com suas evidências/status.
- Indicador de progresso: apresenta a porcentagem de tarefas feitas e quantos critérios foram atendidos no estado mostrado.
- Cards de casos de uso: agrupam o trabalho relacionado ao comportamento do sistema; a contagem mostra critérios atendidos em relação ao total daquele item.
- Botão “Atualizar agora”: solicita uma atualização da análise na interface. “Ao vivo” indica o estado de conexão/atualização do painel, não que os testes estejam sendo executados continuamente.

> **Não interprete o percentual isoladamente**
>
> A porcentagem do dashboard é baseada na evidência estática e no estado do projeto. Para critérios que exigem comportamento real, abra os detalhes da validação dos testes ou execute `testar`.

7. Grafo de dependências entre tarefas

O grafo de dependências responde: “O que bloqueia esta tarefa?” e “O que pode começar agora?”. Cada caixa representa uma tarefa; as linhas mostram relações definidas na especificação. Os IDs e as barras permitem comparar rapidamente o progresso.

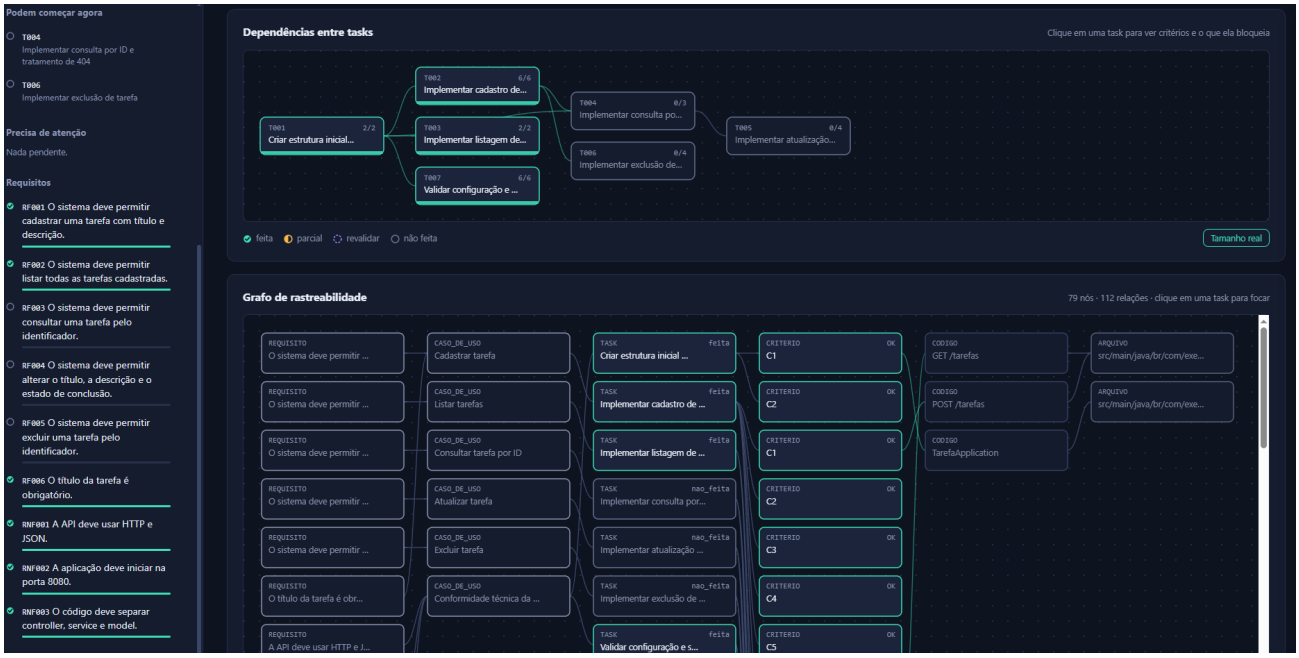


Figura 2 — Grafo de dependências e início do grafo de rastreabilidade.

Como usar o grafo

- Comece pelas tarefas sem dependências pendentes. O comando `python harness.py proxima` mostra uma lista equivalente no terminal.
- Clique em uma tarefa no dashboard para focar nela e consultar seus critérios e bloqueios, conforme indicado na própria interface.
- Se uma tarefa importante estiver bloqueada, conclua ou reavalie a tarefa predecessora em vez de marcar a dependência como concluída manualmente sem evidência.
- Se a relação estiver errada, corrija `depende_de` na especificação e rode `validar` e `sync` novamente.

> **Grafo não substitui a especificação**
>
> A origem oficial da dependência é a especificação. O grafo é a visualização das relações encontradas nela; não altere o desenho esperando que ele mude as regras automaticamente.

8. Grafo de rastreabilidade e stack

O grafo de rastreabilidade conecta os elementos da especificação aos elementos de código. Ele ajuda a responder “por que o sistema considera este critério atendido?” e “que requisitos podem ser afetados quando este arquivo muda?”.

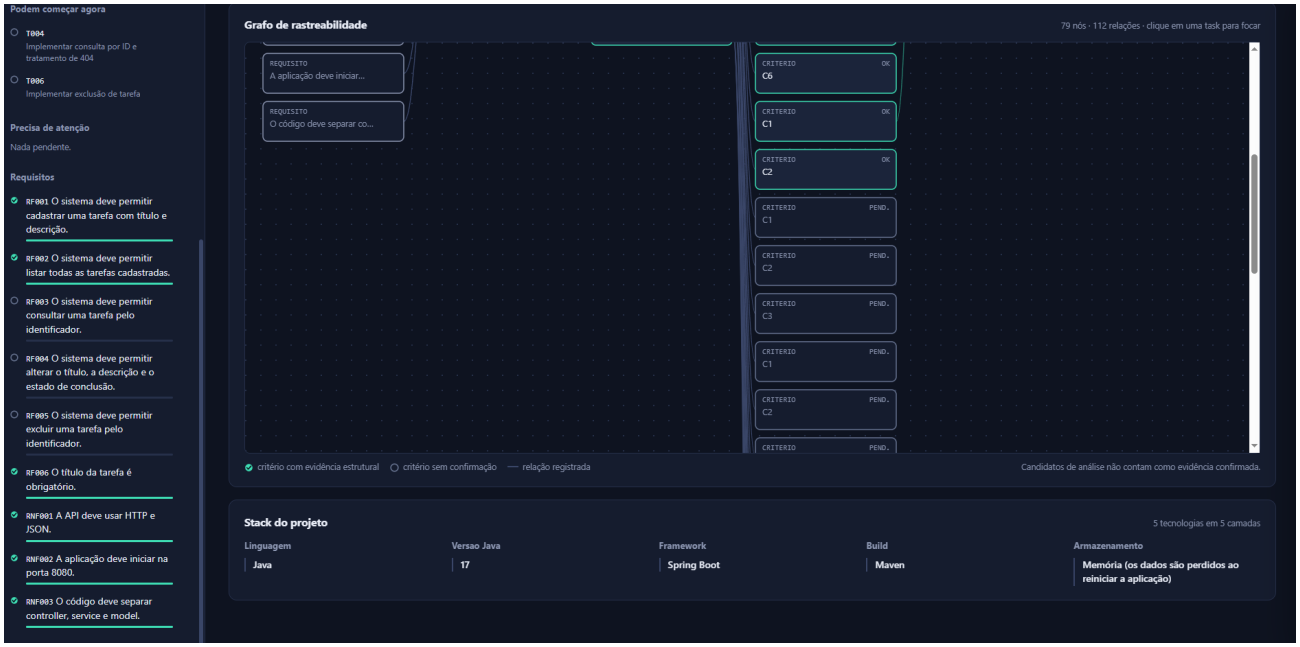


Figura 3 — Detalhe do grafo de rastreabilidade e painel de stack detectada.

Como ler o grafo

- Requisito → caso de uso → tarefa → critério: mostra a trilha de intenção da especificação.
- Critério → código/arquivo: mostra o vínculo com símbolos, classes, funções ou rotas que servem como evidência estrutural.
- As legendas e cores indicam se há evidência estrutural ou se o critério continua sem confirmação. Candidatos sugeridos não são automaticamente prova de implementação.

Stack do projeto

A seção Stack resume as tecnologias que o detector identificou, por exemplo linguagem, framework, ferramenta de build e armazenamento. Ela é uma leitura aproximada dos arquivos do projeto; revise a informação se o repositório contém vários serviços ou ferramentas.

> **Atenção aos números desta captura**
>
> Os 79 nós, 112 relações e a stack Java/Spring Boot que aparecem na imagem pertencem à execução antiga representada pela captura. Eles não descrevem o pacote genérico limpo nem o seu próximo projeto.

9. Revisar e registrar evidências manualmente

A automação sugere relações, mas alguns critérios dependem de contexto que não pode ser decidido apenas por nomes ou expressões regulares. Nesses casos, revise o código antes de registrar o vínculo.

Fluxo seguro de revisão

1. Rode `py harness.py pendentes` para listar os critérios sem confirmação suficiente.
2. Copie um ID de nó candidato real mostrado pelo SoftTracker. Não invente IDs.
3. Use `py harness.py trecho ID_DO_NO` para inspecionar o código daquele nó.
4. Se o nó realmente comprovar o critério, registre o vínculo. Exemplo: `py harness.py link T001/C1 ID_REAL_DO_NO`.
5. Se os candidatos não atendem ao critério e ele está pendente, use `py harness.py nao-feito T001/C1`.
6. Depois, rode `py harness.py status --resumo` e confirme a consequência do julgamento.

Para retirar um julgamento manual e permitir nova avaliação, use `py harness.py unlink T001/C1`. Troque os IDs de exemplo pelos IDs reais da especificação ativa.

Rotina recomendada durante o desenvolvimento

```
py harness.py sync
py harness.py status --resumo
py harness.py proxima
py harness.py go T001
```

Depois de implementar uma tarefa, atualize a análise, revise os critérios que mudaram e confirme se dependências e vínculos ainda fazem sentido. Não use `link` para forçar um status sem ler a evidência.

Vigilância opcional

```
py harness.py watch --intervalo 3
```

O comando `watch` monitora mudanças e sincroniza em intervalos. Encerre com Ctrl+C. Ele não executa automaticamente a suíte comportamental; para isso, use `testar` com a configuração habilitada.

10. Integrar testes comportamentais

A execução comportamental é opcional e fica desabilitada até você configurá-la. O executor aceita comandos em lista de argumentos, roda dentro da pasta do projeto, aplica timeout e espera que o relatório JUnit XML seja emitido na saída padrão. Não use um comando de teste em um projeto que você não considera confiável.

10.1. Vincular um teste ao critério

```
criterios:
  - id: C3
    descricao: "ID inexistente retorna HTTP 404"
    artefato:
      tipo: funcao
      padrao: "buscarPorId"
    verificacao:
      obrigatoria: true
      testes:
        - api.tasks.get_id_missing_returns_404
```

O texto do ID dentro de `verificacao.testes` precisa coincidir com o nome de caso (`testcase name`) que o runner grava no relatório JUnit. Dê nomes estáveis aos testes e associe cada teste somente aos critérios que ele realmente verifica.

10.2. Exemplo de configuração do executor

```
{
  "testes": {
    "habilitado": true,
    "comando": [
      "node", "--test", "--test-reporter=junit",
      "--test-name-pattern={pattern}"
    ],
    "formato": "junit",
    "diretorio": ".",
    "arquivos": ["test/tasks.test.js"],
    "timeout_segundos": 60,
    "env": {}
  }
}
```

Este é um exemplo para projetos Node.js cuja versão do runner ofereça relatório JUnit e filtro por nome. Adapte comando, diretório, arquivo e filtro ao seu runner. O marcador `{pattern}` permite que `testar --impactados` solicite somente os IDs selecionados, mas isso só funciona se o runner interpretar esse argumento como filtro de nomes de teste.

10.3. Executar e conferir

```
py harness.py validar
py harness.py testar
py harness.py testar --json
py harness.py testar --impactados
py harness.py status --json
```

11. Trocar de projeto, segurança e manutenção

Trocar o projeto ativo

Se quiser acompanhar outro projeto usando a mesma instalação, execute novamente `init` com a nova pasta e especificação. A configuração padrão limpa a pasta `estado/` ao detectar a troca de alvo, para não misturar grafos, vínculos manuais e resultados. Isso remove estado gerado, não o código-fonte nem os arquivos de especificação. Se você precisa acompanhar vários projetos ao mesmo tempo, mantenha uma cópia separada do SoftTracker para cada alvo.

Boas práticas de segurança

- Não habilite o executor de testes para repositórios desconhecidos sem revisar o comando e o código que será executado.
- Mantenha `harness.json` e a especificação sob controle de versão quando fizer sentido, mas avalie se caminhos absolutos da máquina devem ser substituídos por caminhos compartilháveis antes de compartilhar a configuração.
- Não marque critérios como atendidos só porque o nome de uma função ou uma rota parece correta. Verifique o conteúdo e, quando necessário, um teste de comportamento.
- Faça backup da especificação e do código-fonte. A pasta `estado/` é resultado gerado e pode ser reconstruída; vínculos manuais podem precisar ser refeitos se o estado for apagado.

12. Solução de problemas

Sintoma	Causa provável / solução
`py` ou `python` não encontrado	Reinstale/ajuste o Python e PATH; confirme com `py --version` ou `python --version`.
Erro ao ler `.yml` / `.yaml`	Instale `PyYAML` com `py -m pip install PyYAML`, ou use JSON.
Nenhuma task reconhecida	A especificação precisa usar `tasks` ou `tarefas_desenvolvimento` e cada tarefa deve ter ID e critérios.
Muitos critérios sem evidência	Ajuste `artefato` e `padrao`, verifique a raiz do projeto e revise os candidatos com `pendentes` e `trecho`.
Painel não abre / porta ocupada	Tente outra porta: `py harness.py painel --porta 8766`; depois abra `http://localhost:8766`.
Executor diz que falta JUnit XML	O comando do runner precisa emitir JUnit XML na saída padrão; crie um adaptador quando o framework só gravar arquivos.
`testar --impactados` não filtra	O comando deve conter `{pattern}` e o runner precisa suportar filtro por nome/ID de teste.
Estado parece pertencer a outro alvo	Pare processos ativos, confirme `harness.json` e reconfigure com `init`. Não reutilize manualmente a pasta `estado/` de outro projeto.

13. Checklist de ponta a ponta

Use esta lista quando configurar um novo alvo ou antes de considerar o rastreamento pronto para a rotina de desenvolvimento.

- ☐ O pacote SoftTracker foi extraído em uma pasta própria, separada do projeto-alvo.
- ☐ Python funciona no terminal e PyYAML está instalado se a especificação for YAML.
- ☐ O diretório configurado contém apenas o projeto que será monitorado.
- ☐ A especificação descreve o mesmo projeto, usa IDs estáveis e define critérios observáveis.
- ☐ `python harness.py init --projeto ... --spec ...` concluiu e criou `harness.json`.
- ☐ `python harness.py validar` terminou sem erros.
- ☐ `python harness.py sync` e `python harness.py status --resumo` mostram os dados do alvo correto.
- ☐ O dashboard abre e os nomes/contagens correspondem à especificação do projeto ativo.
- ☐ As dependências fazem sentido e a lista `proxima` reflete o que pode começar.
- ☐ Os critérios com evidências duvidosas foram revisados usando `pendentes`, `trecho` e, quando necessário, `link`/`nao-feito`.
- ☐ Se precisar comprovar comportamento, testes reais foram vinculados aos critérios, o executor foi habilitado deliberadamente e o relatório JUnit foi validado.
- ☐ Depois de mudar o código, rodei `sync`/`status` e reexecutei os testes desatualizados.

> **Regra de ouro**
>
> O grafo ajuda a encontrar relações e reduzir o trabalho repetido; ele não substitui a especificação nem os testes. Considere uma funcionalidade realmente verificada somente quando as evidências estão corretas e os testes exigidos passaram com resultados ainda atuais.

Referências úteis

Python para Windows: https://www.python.org/downloads/windows/
Documentação do Python: https://docs.python.org/3/
Para comandos e opções adicionais do SoftTracker, execute `py harness.py --help` e `py harness.py testar --help` na pasta da ferramenta.